

Guia de Revisão - JavaScript Básico

Panorama completo, ordem de estudo e versão didática dos códigos

Material organizado a partir do zip enviado

Objetivo: deixar tudo mais limpo para revisão rápida, estudo recorrente e consulta por tema.

1. Como usar este guia

Use este material em três camadas:

- primeiro leia o panorama por assunto para entender o que já foi aprendido;
- depois consulte a tabela arquivo por arquivo para ligar cada conteúdo ao nome do script;
- por fim, revise os exemplos de código comentados para fixar a lógica.

A ordem abaixo já está pensada para ir do mais básico ao mais prático.

2. Ordem de revisão recomendada

Ordem	Tema	Para revisar
1	Variáveis	var, let, const, nomeação e escopo
2	Tipos de dados	string, number, boolean, null, undefined
3	Strings e saída	concatenação, crase e template literals
4	Conversão e coerção	Number(), toString(), typeof e conversão automática
5	Operadores	aritméticos, comparação e lógicos
6	Precedência	ordem de execução das expressões
7	Incremento e decremento	++, --, += e -=
8	Math	funções matemáticas básicas
9	Condicionais	if, else if, else, ternário, switch
10	Repetição e funções	for, while, function, arrow, callback, return
11	Erros	try, catch, finally, throw

3. Panorama completo do que foi aprendido

Variáveis e escopo

- Diferença entre var, let e const.
- Reatribuição de valores e comportamento de declaração.
- Boas práticas de nomeação: nomes claros, sem acentos e com case consistente.

Tipos de dados e texto

- String, number e boolean.
- Uso de aspas simples, duplas e crase.
- Concatenação com + e interpolação com template literals.
- Quebra de linha com crase.

Conversão e coerção

- Conversão explícita com Number() e toString().
- Leitura do tipo com typeof.
- Coerção automática em expressões como "10" + 5.

Operadores

- Aritméticos: +, -, /, *, ** e %.
- Comparação: ==, ===, !=, !==, >, <, >=, <=
- Lógicos: &&, || e !.
- Uso do resto para descobrir paridade.

Precedência e incremento

- Ordem de prioridade dos operadores.
- Pós e pré-incremento.
- Pré-decremento.
- Atribuições compostas como +=.

Math

- abs, ceil, floor, round, trunc.
- pow, sqrt, min, max, random, PI e sin.

Condicionais

- if, else if e else.
- Operador ternário.
- Truthy e falsy.
- switch e break.
- Tratamento de exceções com try, catch, finally e throw.

Repetições e estruturas de dados

- for com índice.
- for...of para iterar arrays/strings.
- for...in para objetos.
- while.
- Uso de arrays e objetos como suporte para os laços.

Funções

- function declarada, função anônima e arrow function.
- Parâmetros e argumentos.
- return para devolver resultado.
- Funções reutilizáveis.
- Callback como função passada para outra função.
- Exercícios práticos com números, tabuada e hora.

Fora do conteúdo principal

- Existe uma pasta DOM, mas sem arquivos de código nela.
- Isso indica que o assunto foi reservado, mas não apareceu no zip enviado.

4. Mapa dos arquivos do zip

A tabela abaixo liga cada arquivo ao que ele realmente ensina e aponta o que vale revisar com mais atenção.

Arquivo	Aprendizado	Observação
variaveis/variaveis.js	var, let, const; escopo; boas práticas de nomes	Base de todo o resto. Aqui você aprendeu a declarar variáveis e a pensar em escopo.
tipos de dados/index.js	String, number, boolean, concatenação, template literals	Bom arquivo para revisar texto e saída formatada.
tipos de dados/conversao.js	Conversão explícita, typeof e coerção	Importante para entender erros e comportamento automático do JS.
expressoes-operadores/index.js	Operadores aritméticos e paridade	Aqui começa a lógica matemática usada em problemas.
ordem-de-precedência/index.js	Prioridade dos operadores	Serve como referência rápida para não errar expressões complexas.
incremento-Decremento/index.js	++, --, +=, -=, prefixo e pós-fixado	Muito útil para laços e contadores.
operadores-logicos/index.js	==, ===, !=, !==	Ótimo para comparação e evitar bugs de tipo.
operadores-logicos2/index.js	&&, e !	Base de validação de condições.
condicionais/index.js	Ternário, falsy e truthy	Ajuda a simplificar decisões curtas.
condicionais/if,else,elseif.js	if, else if, else	A estrutura principal de decisão.
condicionais/switch.js	switch, case, break e default	Boa para menus e escolhas fixas.
condicionais/tratamento-exceções/index.js	try, catch, finally e throw	Mostra tratamento de erros e fluxo de exceção.
Estruturas-de-repetição/for.js	for, for...of e for...in	Aqui você aprende a percorrer listas e objetos.
Estruturas-de-repetição/while.js	while	Exercício de repetição com condição.
funções/função.js	funções, parâmetros, argumentos e return	Esse é um dos arquivos mais importantes para fixar.
funções/funcao-anonima.js	função anônima e arrow function	Bom para entender sintaxe moderna.
funções/callbackFunction.js	callback	Mostra funções sendo passadas como argumento.
funções/calculadora.js	função com switch e cálculo	Exercício prático de organização lógica.
funções/desafios/index.js	exercícios de fixação	Treino de laços, funções e datas.
mathss/math.js	Math.*	Lista de consulta rápida dos métodos matemáticos.

5. Códigos comentados e mais didáticos

Abaixo estão versões mais limpas dos pontos principais. A ideia é mostrar a lógica de forma que sirva como revisão rápida.

Variáveis

Versão comentada para revisão:

```
// var: escopo mais amplo e pode ser redeclarada
var nome = "Leonel";

// let: pode mudar o valor, mas não deve ser redeclarada no mesmo bloco
let idade = 20;
idade = 21;

// const: valor fixo após a atribuição
const anoNascimento = 2005;

console.log(nome, idade, anoNascimento);
```

Tipos de dados e texto

Versão comentada para revisão:

```
const nome = "Leonel";
const telefone = 88993393;

console.log(typeof nome);      // string
console.log(typeof telefone);  // number

// Concatenação
console.log("Meu nome é " + nome + " e meu telefone é " + telefone);

// Template literal: mais legível
console.log(`Meu nome é ${nome} e meu telefone é ${telefone}`);
```

Conversão e coerção

Versão comentada para revisão:

```
const valorTexto = "9";
const valorNumero = Number(valorTexto); // conversão explícita

console.log(typeof valorNumero); // number
console.log(String(18));         // "18"

// Coerção automática: o JS escolhe um tipo que faça sentido
console.log("10" + 5); // "105"
```

Operadores

Versão comentada para revisão:

```
console.log(2 + 2); // soma
console.log(20 - 2); // subtração
console.log(20 / 2); // divisão
console.log(2 * 2); // multiplicação
console.log(10 ** 2); // potência
console.log(20 % 2); // resto da divisão

// Paridade
```

```
const numero = 20;
console.log(numero % 2 === 0); // true se for par
```

Condicionais

Versão comentada para revisão:

```
const nota = 85;

if (nota >= 90) {
  console.log("Aprovado com excelência!");
} else if (nota >= 60) {
  console.log("Aprovado!");
} else {
  console.log("Reprovado.");
}

// Ternário
console.log(nota >= 60 ? "passou" : "não passou");
```

Switch

Versão comentada para revisão:

```
const opcao = 2;

switch (opcao) {
  case 1:
    console.log("Consultar pedido");
    break;
  case 2:
    console.log("Falar com atendente");
    break;
  default:
    console.log("Opção inválida");
}
```

Exceções

Versão comentada para revisão:

```
try {
  console.log(resultado);
} catch (error) {
  console.log("Houve um erro:", error.message);
} finally {
  console.log("Fim da execução");
}

const numero = 0;
if (numero === 0) {
  throw new Error("O valor não pode ser zero");
}
```

Laços

Versão comentada para revisão:

```
const alunos = ["Vitor", "Bruno", "Bruna"];

// for clássico: usa índice
for (let i = 0; i < alunos.length; i++) {
  console.log(alunos[i]);
}

// for...of: percorre valores
for (const aluno of alunos) {
  console.log(aluno);
}
```

```
// for...in: percorre chaves de um objeto
const aluno = { nome: "Leo", idade: 20 };
for (const chave in aluno) {
  console.log(chave, aluno[chave]);
}

// while: executa enquanto a condição for verdadeira
let i = 5;
while (i > 0) {
  console.log(i);
  i--;
}
```

Funções

Versão comentada para revisão:

```
function soma(a, b) {
  return a + b;
}

const mensagem = function () {
  console.log("Olá!");
};

const somarArrow = (a, b) => a + b;

console.log(soma(4, 5));
mensagem();
console.log(somarArrow(10, 2));

// callback: função passada como argumento
function executar(tarefa, callback) {
  console.log("Executando:", tarefa);
  callback();
}
```

Math

Versão comentada para revisão:

```
console.log(Math.abs(-5)); // 5
console.log(Math.ceil(4.1)); // 5
console.log(Math.floor(4.9)); // 4
console.log(Math.round(4.5)); // 5
console.log(Math.trunc(4.9)); // 4
console.log(Math.sqrt(25)); // 5
console.log(Math.max(1, 7, 2)); // 7
console.log(Math.min(1, 7, 2)); // 1
```

6. O que ficou implícito e vale adicionar ao panorama

- Arrays aparecem em for...of e nos exercícios, então vale tratá-los como parte do estudo, mesmo sem um arquivo exclusivo.
- Objetos também aparecem no for...in, então já houve contato com a estrutura, ainda que bem básico.
- Date aparece como ideia nos desafios de funções, mas não chegou a virar conteúdo completo no zip.
- A pasta DOM existe, porém sem código dentro. Portanto, DOM não entra como aprendizado concluído desse material.

7. Estrutura de pastas sugerida para revisão

Se você quiser manter o material arrumado, uma organização melhor seria algo assim:

```
Javascript-basico/  
├── 01-variaveis-e-tipos/  
│   ├── variaveis.js  
│   ├── tipos-de-dados.js  
│   └── conversao.js  
├── 02-operadores-e-precedencia/  
│   ├── operadores-aritmeticos.js  
│   ├── operadores-logicos.js  
│   ├── operadores-logicos-2.js  
│   └── precedencia.js  
├── 03-controle-de-fluxo/  
│   ├── if-else.js  
│   ├── ternario.js  
│   ├── switch.js  
│   └── excecoes.js  
├── 04-repeticao/  
│   ├── for.js  
│   └── while.js  
├── 05-funcoes/  
│   ├── declaradas.js  
│   ├── anonimas.js  
│   ├── arrow.js  
│   ├── callback.js  
│   ├── calculadora.js  
│   └── desafios.js  
└── 06-built-ins/  
    └── math.js
```

8. Resumo final

O que você já construiu aqui é uma base real de JavaScript básico: tipos, operadores, decisões, laços, funções e funções utilitárias da linguagem.

O próximo passo natural, depois de revisar isso com calma, é aprofundar arrays, objetos, funções com mais de um retorno, DOM, eventos e manipulação de formulários.